

HW 2: Time Command

Task 1. Implement a time1 command that reports elapsed time.

I implemented a new xv6 user command called time1 for this task, which calculates and outputs the amount of CPU time that a program requires to execute. This necessitated knowledge of xv6 parent-child synchronization, system calls, and process generation.

```
$ time1 matmul
Time: 19 ticks
elapsed time: 19 ticks
$ time1 matmul & time1 matmul &
$ Time: 37 ticks
elapsed time: 38 ticks
Time: 36 ticks
elapsed time: 37 ticks

$ time1 sleep 10
elapsed time: 10 ticks
$
```

Some difficulties that I had come across were in the make file and with the additional two C files. As I didn't place those two files under the user directory but under kernel, and some changes to the Makefile.

Task 2. Keep track of how much CPU time a process has been used.

The changes that I did were made in the kernel directory on proc.c, proc.h and trap.c files.

The changes in both proc files with the variable keeping track of the CPU time consumed by each process done, and the other making sure that starts at 0. The difficulties that I came across were just looking where to make the changes in those files for the CPU time tracking.

By making these changes, xv6 is able to determine how much CPU time each process uses. Your time1 application and upcoming scheduling enhancements will then use this information.

Task 3. Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

For this task modified pstat.h that will store the CPU usage from each process information.

User.h to declare the wait2() system call so it can be used in user space programs and be linked

to the kernel. `Usys.pl` to call the `wait2()` and change to the kernel. For `syscall.h` and `c` assign an identifier to `wait2` and the other maps the `wait2` syscall number. `Sysproc.c` what was change was adding a function for `wait2` to retrieve the user-space pointer and calls the kernel for `wait2()` function. In `proc.c` was implement the `wait2()` function by copying the `wait()` and changing it by it returning both child's exit status. And `defs.h` making the function visible to the other kernel.

Some of the difficulties that I came to was just looking at were to make the changes and to implement it.

Task 3 taught me how to change both kernel and user-level files in order to develop a new system call in `xv6`. The user-space declaration in `user.h`, the addition of entries in `usys.pl`, `syscall.h`, and `syscall.c`, and the final implementation of the kernel-side logic in `sysproc.c` and `proc.c` were all steps in this effort that improved my understanding of how the system call mechanism operates. I learned how to properly move data between user and kernel space by managing user-space pointers with `argaddr()`, `copyin()`, and `copyout()`. I also learned how to use a custom structure (`struct rusage`) to track and return process information, including CPU time. All things considered, this assignment improved my comprehension of kernel-user interactions and `xv6` process management while providing me with useful insight into how operating systems reveal internal process data through system calls.

Task 4. Implement a `time` command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

Your `time.c` code should be in the user directory in the `hw2` branch of your `xv6` repo. Remember to add your new command to `UPROGS` in `Makefile`.

Show results from running your `time` command.

Extra Credit (5 points). Discuss limitations of our `time` command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).